

CSCE 701 CYBERSECURITY · MODULE 11 · LECTURE NOTES

Human Factors

A control people route around is not a control.

MODULE OUTCOMES

- Explain why usability is a security property.
- Explain the compliance budget and its consequences.
- Design phishing defences that do not rely on vigilance.
- Explain secure defaults and why they dominate.
- Review a control for the ways people will avoid it.

1 Usability is a security property

AN UNUSABLE CONTROL IS NOT A STRONG CONTROL — IT IS AN ABSENT ONE

A password policy that people satisfy by writing their passwords on a note has *reduced* security (Module 02 §2) — and the policy looks stronger on paper than the one it replaced.

So usability and security are not opposed quantities. The usability of a control determines whether it is actually in place, which makes it **a component of the control's strength rather than a cost to be traded against it** — and that reframing is the whole content of this module.

And the evidence is that people behave rationally. They route around controls whose cost to them exceeds the benefit they perceive, which is a sensible response to a badly-designed control rather than a failure of discipline — and treating it as a discipline problem guarantees the design is not fixed.

So the design question is 'what will people actually do' rather than 'what should they do' — which is the same empirical turn as CSCE 671's, and is the reason these two courses share an argument despite being in different semesters.

THE COMPLIANCE BUDGET IS FINITE AND YOU ARE SPENDING IT

Each person has a limited willingness to absorb security friction — Beautelement, Sasse and Wonham's compliance budget — and **every control you impose spends from the same budget**.

So a mandatory annual training module, a complex password rule, a VPN that drops twice a day, and a three-day access approval process all compete with one another — and **spending the budget on low-value controls means the high-value one is the one that gets ignored**, because by then there is nothing left.

Which explains a pattern that is otherwise puzzling: an organisation with a great many controls and demonstrably poor security, because the budget was exhausted long before anything important was asked of anyone.

So remove controls deliberately. Dropping three low-value requirements in order to buy genuine compliance with one high-value one is a net gain — and it is a decision almost nobody makes, because removing a control looks like weakening security and is hard to defend in an audit. **Which is a measurement problem (Module 12) rather than a disagreement about the facts.**

2 Phishing

DESIGN SO THAT BEING PHISHED DOES NOT MATTER

Phishing awareness training reduces click rates and does not eliminate them — the measured effect is real and modest — and **a sufficiently well-crafted lure will catch a careful person on a bad day**, which is not a training deficiency.

So the goal is not a zero click rate but a *harmless* click. Origin-bound second factors mean a phished credential is simply unusable by the attacker (Module 02 §1) — which removes the entire attack rather than reducing its frequency, and is the single most effective anti-phishing control available.

And the supporting controls are all consequence-reducing: least privilege, so that a stolen account yields little; segmentation, so that it does not spread (Module 04 §2); and detection, so that its use is noticed (Module 09 §2).

Which means phishing is an authentication and architecture problem rather than a training problem — and **treating it as a training problem places the burden on the person least able to carry it**, while leaving the architecture that made the consequence severe entirely unchanged.

- **A one-click report button** that reaches someone who acts on it — **because a single reported phish lets you find every other recipient and pull the message**, which is worth far more than the one person's caution.
- **Thank the reporter, every time, including for false alarms** — **the cost of a false report is one minute of someone's time and the cost of a silent true one is an incident**. The asymmetry is enormous and the incentive should reflect it.

- **And never penalise clicking. A person who clicked and is afraid to say so is the worst possible outcome**, because the attacker now has hours of undetected access — **and it is exactly what a punitive simulated-phishing programme produces.**
- **Which is Module 10 §4's blamelessness, applied before the incident rather than after it** — the same information-gathering argument, at a different point in the timeline.
- **And the metric should be the reporting rate rather than the click rate — the first is the thing you want to increase**, and **measuring clicks incentivises punishing clickers, which suppresses reporting** and makes the organisation less safe while the metric improves (Module 12's subject exactly).

3 Defaults

THE DEFAULT IS THE SETTING ALMOST EVERYONE HAS

Only a small minority of users change any given setting, across essentially every system ever measured — so the default is, in practice, the configuration of your system as deployed.

Which makes secure-by-default worth more than any amount of documentation, guidance, hardening guide, or training — and it is the one intervention that requires no user action whatsoever, so it spends nothing from §1's compliance budget.

And the historical examples are large: HTTPS by default and then HTTPS-preferred in browsers, SameSite cookies defaulting to Lax (Module 06 §3), automatic updates, full-disk encryption on by default on phones, and randomised hash seeds in language runtimes (Module 01 §1) — each eliminated a vulnerability class at population scale without asking anyone to do anything.

So if you ship software, the highest-leverage security work available to you is changing a default — and the cost is usually a backwards-compatibility argument rather than an engineering one, which means the obstacle is organisational and the fix is available.

4 Reviewing a control

- **What is the cheapest way to avoid this? Somebody will find it** — a shared account, a personal device, a standing exception, a screenshot of the data, an export to a spreadsheet — **and the avoidance path becomes the real process** while the control remains on the diagram.
- **How often must a person comply, and what does it cost them each time? Daily friction is spent from the compliance budget (§1) and is remembered and resented;** an annual cost is nearly free by comparison.
- **What happens when it fails? If a failure blocks work, there will be a documented workaround within a month** — and **the workaround will not be reviewed**, which makes it a worse control than the one it replaced.
- **Does it require sustained vigilance? If the control is 'notice something unusual', it will fail**, because **sustained vigilance is not a human capability** and designing around the assumption that it is guarantees the failure.
- **And could a default, an architecture change, or a tool perform this instead of a person? This is the question to ask first** — a control that a person must perform is the weakest kind, and moving it into the system is almost always possible and almost never attempted.

WHAT THIS MODULE ADDS TO THE REST OF THE COURSE

Every control in Modules 02 through 09 is implemented by people and avoided by people — so a design that ignores that is incomplete rather than merely optimistic, and the incompleteness is predictable in advance from §4's questions.

And the direction of the fix is consistent across every case: move the control into the system, into the default, or into the architecture, and away from a person's ongoing attention.

Which is also the most reliable way to make security cheaper — a control that requires no ongoing human effort has no ongoing cost and does not degrade over time, whereas one that depends on attention degrades from the day it is introduced.

So 'who has to do something for this to work, and how often' is the most useful single question to ask of a security design — and the best available answer is 'nobody, never'.

Where to see this taught differently

Original material only goes so far — hearing the same idea in another voice is how it sticks. These are free, and they cover this module specifically.

Resource	What it covers for this module
Anderson — Security Engineering, the psychology and usability chapters (free PDF)	§1 and §2 with the evidence — free in full, and the strongest treatment of this material anywhere.
Beautement, Sasse & Wonham — The Compliance Budget (free)	§1's second callout, in the original — and the framing is more useful than any individual finding.
Adams & Sasse — Users Are Not the Enemy (free)	§1's argument, from 1999 , and the title is the thesis. It took the industry twenty years to act on it.
Thaler & Sunstein — Nudge; and the default-effect literature	§3's effect, outside security — where the default-dominance finding was established, and it transfers directly.

Exercises

Do these in code, not on paper, unless the exercise says otherwise. The module is not finished until they run.

- Find a control in your organisation that people route around**, and document how.
- Estimate the daily friction** your controls impose on one person.
- Identify three low-value controls** you could remove to buy compliance with one high-value one.
- Check whether your organisation uses origin-bound second factors**, and what the obstacle is if not.

- 5 **Find your phishing reporting path** and time how long it takes to use.
- 6 **Determine whether your programme measures clicks or reports**, and what it does to clickers.
- 7 **List the security-relevant defaults** in software you ship, and whether each is the safe one.
- 8 **Change one default to the safe value** and note the compatibility objection.
- 9 **Run §4's five questions** against one of your controls.
- 10 **Find a control that requires vigilance** and replace it with one that does not.

Self-check

Answer these in writing, from memory, without looking back at the notes. If you cannot, you have read this module but not yet learned it — that is useful information, not a failure.

- 1 Why is an unusable control an absent one?
- 2 Why is routing around a control rational?
- 3 State the compliance budget and what it explains.
- 4 Why is removing a control sometimes a net gain, and why is it rare?
- 5 Why is phishing not a training problem, and what is the effective control?
- 6 Why measure reporting rate rather than click rate?
- 7 Why do defaults dominate, and give four historical examples.
- 8 Give five questions to ask of a control, and which comes first.
- 9 What is the consistent direction of the fix, and the best answer to 'who has to do something'?